

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 12

Producer, Consumer and the Other Tools

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reiţman, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. Producer and consumer **2**
2.1 Observations 3

3 Part 2. The condition variable **4**
3.1 Observations 5

4 Part 3. Readers and writers **6**
4.1 Observations 7

5 Part 4. Build a deadlock, then remove it **8**
5.1 Observations 9

6 Conclusion **10**

1 Goal of the work

In the previous laboratory work one semaphore was used at a time: as a lock, as a signal and as a counter. In this work the synchronization tools work together. The goal is to solve the two classic problems of concurrency, producer–consumer and readers–writers, to use a condition variable, and to build a deadlock and then remove it.

The work was performed in an Ubuntu 24.04 (arm64) container running on Docker Desktop on macOS, with 8 CPUs visible to the container, kernel 7.0.14-linuxkit and gcc 13.3.0. Every program was compiled with `gcc -Wall -pthread -o NAME NAME.c`.

2 Part 1. Producer and consumer

A producer puts 8 items into a buffer of 4 slots, and a slow consumer takes them out. Three tools work together: the semaphore `empty` counts the free slots (it starts at 4), the semaphore `full` counts the filled slots (it starts at 0), and a mutex protects the buffer itself.

Listing 1: `prodcons.c`

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE  4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;     /* count only for printing */
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);               /* wait for a free slot */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put %d (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full);                /* one more filled slot */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);                /* wait for something to take */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("    take %d (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
```

```
        sem_post(&empty);               /* one more free slot */
        usleep(300000);                 /* the consumer is slow */
    }
    return NULL;
}

int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Listing 2: Output of `prodcons`

```
$ gcc -Wall -pthread -o prodcons prodcons.c
$ time ./prodcons
put 1 (in buffer: 1)
put 2 (in buffer: 2)
put 3 (in buffer: 3)
put 4 (in buffer: 4)
    take 1 (in buffer: 3)
put 5 (in buffer: 4)
    take 2 (in buffer: 3)
put 6 (in buffer: 4)
    take 3 (in buffer: 3)
put 7 (in buffer: 4)
    take 4 (in buffer: 3)
put 8 (in buffer: 4)
    take 5 (in buffer: 3)
    take 6 (in buffer: 2)
    take 7 (in buffer: 1)
    take 8 (in buffer: 0)

real    0m2.421s
```

The producer fills the 4 slots at once. After that, every `take` frees one slot and the producer fills it again immediately. The run takes about 2.4 s: 8 items, each followed by the 0.3 s pause of the consumer.

2.1 Observations

Question 1. The producer is fast, yet "in buffer" never goes above 4. Which call stops it? The call `sem_wait(&empty)` in `producer()`. The semaphore `empty` starts at `SIZE` = 4 and counts the free slots. After four `puts` it is 0, so the fifth `sem_wait(&empty)` blocks. The producer continues only after the consumer takes an item and calls `sem_post(&empty)`.

Question 2. Who waits at the start, and which call makes it wait? The **consumer**. It is created first, but `full` starts at 0 (there is nothing to take), so its

`sem_wait(&full)` blocks. It wakes up when the producer puts the first item and calls `sem_post(&full)`.

Question 3. The mutex is locked before `sem_wait(&empty)`. What happens when the buffer is full, and why? I swapped the first two lines of the loop in `producer()`:

```
        pthread_mutex_lock(&lock);
        sem_wait(&empty);               /* wait for a free slot */
```

Listing 3: Output of the swapped version

```
$ timeout 5 ./prodcons_swap; echo "exit code: $?"
put 1 (in buffer: 1)
put 2 (in buffer: 2)
put 3 (in buffer: 3)
put 4 (in buffer: 4)
exit code: 124
```

The program **deadlocks** when the buffer is full and must be killed by `timeout` (exit code 124). For the fifth item the producer first takes the mutex, then sleeps in `sem_wait(&empty)` because there is no free slot, **while still holding the mutex**. The only thread that can free a slot is the consumer, but before it takes an item it needs `pthread_mutex_lock(&lock)`, which the producer holds. The producer waits for the consumer and the consumer waits for the producer, so neither of them can ever continue.

In this run the consumer did not manage to take even one item: every time the producer released the mutex, it took it again at once for the next item, before the consumer, which had just been woken up, could get it. The producer stopped only when the buffer was full, while holding the lock. The rule is: **never wait on a semaphore while holding a mutex** that the other side needs in order to post that semaphore.

3 Part 2. The condition variable

The problem of `order.c` from Lab 11 (B must read only after A has written) is solved here in a different way. The state is a normal variable, `ready`, protected by a mutex. The condition variable is only the place where B sleeps until A announces that something has changed. `pthread_cond_wait` releases the mutex, sleeps, and takes the mutex again before it returns.

Listing 4: `condvar.c`

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <unistd.h>
4  #include <pthread.h>
5
6  char message[32];
7  int ready = 0;                          /* the condition itself */
8  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
9  pthread_cond_t changed = PTHREAD_COND_INITIALIZER;
10
11 void *writer(void *arg) {
12     sleep(1);
13     pthread_mutex_lock(&lock);
```

```c
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);      /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) {                    /* while, not if: re-check after waking */
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock);   /* unlocks, sleeps, locks again */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

Listing 5: Output of `condvar`

```
$ gcc -Wall -pthread -o condvar condvar.c
$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

3.1 Observations

Question 1. Which variable remembers that the message is ready? The integer `ready`, protected by the mutex `lock`. A condition variable has no memory: `pthread_cond_signal` only wakes a thread that is already sleeping, and if nobody is waiting the signal is lost. This is why the state must be kept in a normal variable, unlike the semaphore of `order.c`, whose value itself remembered the `sem_post`.

Question 2. Does it still work with `if (!ready)`? I replaced `while (!ready)` with `if (!ready)` and ran the program three times. All three runs printed the same output:

```
$ ./condvar_if
B: not ready, I sleep
A: message written
B: I read "hello"
```

Yes, it still works *here*. There is only one writer, it sets `ready = 1` before it signals, and it signals only once, so when B wakes up `ready` is always 1.

Question 3. Then why do we write `while`? A thread can return from `pthread_cond_wait` while the condition is still false:

- **Spurious wakeups:** POSIX allows `pthread_cond_wait` to return even though nobody called `signal`, for example after an interrupted system call inside the implementation.
- **Another thread got there first:** with several waiters (or `pthread_cond_broadcast`), between the signal and the moment the woken thread takes the mutex again, another thread can take the mutex and use or change the state, for example set `ready` back to 0.

With `while`, the thread checks the condition again after every wakeup and goes back to sleep if it is not true; with `if` it would continue with a false condition.

4 Part 3. Readers and writers

Many readers may read at the same time, but a writer must be alone. Two semaphores are used: `room` (1 = free for a writer) and `rlock` (a lock around the counter `readers`). The first reader to enter locks the room, and the last one to leave unlocks it.

Listing 6: `readers.c`

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;                  /* the shared data */
int readers = 0;                /* how many are reading now */
sem_t rlock;                    /* protects the counter readers */
sem_t room;                     /* 1 = the room is free for a writer */

void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)         /* the first reader in locks out the writers */
        sem_wait(&room);
    sem_post(&rlock);

    printf("reader %ld reads %d (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)         /* the last reader out lets the writers in */
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);            /* alone in the room */
```

```
    value += 10;
    printf("writer %ld writes %d (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}

int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);
    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);
    for (int i = 0; i < READERS; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++) pthread_join(w[i], NULL);
    return 0;
}
```

Listing 7: Output of `readers` (the same in all three runs)

```
$ gcc -Wall -pthread -o readers readers.c
$ ./readers
writer 1 writes 10 (alone)
reader 1 reads 10 (readers inside: 1)
reader 2 reads 10 (readers inside: 2)
reader 3 reads 10 (readers inside: 3)
writer 2 writes 20 (alone)
```

4.1 Observations

Question 1. Which two lines of `reader()` allow three readers together but a writer always alone?

```
    if (++readers == 1)  sem_wait(&room);   /* the first reader in locks out the
    writers */
    if (--readers == 0)  sem_post(&room);   /* the last reader out lets the writers in
     */
```

Only the **first** reader takes `room`, on behalf of the whole group of readers; the readers that come after it do not touch `room`, so they enter together. A writer needs `room` for itself (`sem_wait(&room)`), so it cannot enter while any reader is inside, and nobody can enter while a writer holds it. Only the **last** reader to leave gives `room` back.

Question 2. Why does the counter `readers` need its own lock? `++readers` and `--readers` are read–add–write in three steps, exactly like `counter++` in Lab 11, so two readers can interleave them. Without `rlock`:

- two readers arrive together, both read `readers = 0` and both write 1; both think they are the first and both call `sem_wait(&room)`, and the second one blocks forever, because the first already took `room`;
- an update is lost and the count becomes too small: `readers` reaches 0 while a reader is still inside, `room` is posted, and a writer enters while that reader is still reading;

- the count becomes too large: `readers` never returns to 0, `room` is never released and the writers wait forever.

`rlock` also makes the test `== 1` / `== 0` and the matching `sem_wait`/`sem_post(&room)` one indivisible step.

Question 3. Readers keep arriving without a pause. What happens to writer 2? Writer 2 **waits forever**. `room` is released only when `readers` drops back to 0. If a new reader always arrives before the last one leaves, `readers` never becomes 0, `sem_post(&room)` is never called, and writer 2 stays blocked in `sem_wait(&room)`, although the program as a whole keeps running. This is **starvation**: a thread that is ready to run but never receives the resource. This solution gives priority to the readers; a fair version would stop new readers from entering once a writer is waiting.

5 Part 4. Build a deadlock, then remove it

Two threads take two mutexes in opposite order: `one()` takes A, then B, and in the original version `two()` takes B, then A.

Listing 8: The thread functions of the original `deadlock.c`

```c
void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);             /* waits for two */
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&B);             /* the fix: lock A first here too */
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&A);             /* waits for one */
    printf("two: I have both\n");
    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);
    return NULL;
}
```

Listing 9: Output of the original version

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ time (timeout 5 ./deadlock; echo "exit code: $?")
one: I have A, I want B
two: I have B, I want A
exit code: 124

real    0m5.001s
```

The two lines appear, then nothing happens for 5 seconds, then `timeout` kills the program.

The fix. In `two()` the two `pthread_mutex_lock` lines and the two `pthread_mutex_unlock` lines were swapped, so both threads take A first and B second. The submitted `deadlock.c` is this version:

Listing 10: `two()` in the fixed `deadlock.c`

```c
void *two(void *arg) {
    pthread_mutex_lock(&A);             /* the fix: lock A first here too */
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&B);             /* waits for one */
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}
```

Listing 11: Output of the fixed version

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 ./deadlock; echo "exit code: $?"
one: I have A, I want B
one: I have both
two: I have B, I want A
two: I have both
done
exit code: 0
```

In the fixed version the message `two: I have B, I want A` is no longer literally true (at that moment `two` holds A and wants B); the original `printf` text was kept so that the output matches the one in the laboratory handout.

5.1 Observations

Question 1. Which thread holds which mutex, and which one is each thread waiting for? Thread `one` holds **A** and waits for **B**; thread `two` holds **B** and waits for **A**. The waits form a circle (Figure 1), so no thread can continue and none of them will ever release what it holds.

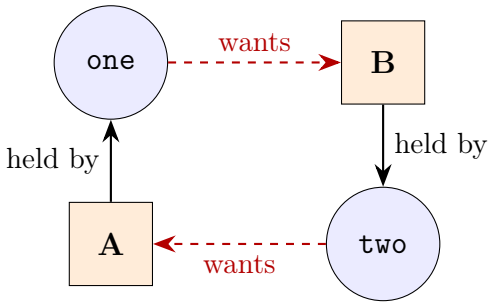


Figure 1: The circle of waits in the deadlock

Question 2. Without the two `sleep(1)` lines: does it still always hang? Why is that worse? I removed both `sleep(1)` lines from the original version and ran it ten times, each time with `timeout 2`:

Run	1	2	3	4	5	6	7	8	9	10
Result	**hang**	done	done	**hang**	done	done	done	done	done	done

It no longer hangs every time: 2 of the 10 runs hung (exit code 124) and 8 finished normally. Over 1000 runs, **332** hung. Without the pause, one thread often takes both locks and finishes before the other thread starts, so the circle does not form. This is **worse, not better**: the bug is still there, but it now appears only sometimes and at random. The program passes most tests, then hangs in front of a user, and the problem is hard to reproduce and to debug. The `sleep(1)` made the deadlock certain and therefore easy to see.

Question 3. Why can taking the locks in the same order never deadlock? If every thread takes A before B, a thread that holds B already holds A, so no thread can ever hold B while waiting for A, and the circle of waits cannot form.

6 Conclusion

This laboratory work combined the synchronization tools of the previous one. In the producer–consumer problem two counting semaphores (`empty` and `full`) control *when* each side may act, and a mutex protects the buffer itself; taking the mutex before waiting on a semaphore turned a correct program into a deadlock. The condition variable showed a different style of waiting: the state lives in a normal variable, and the condition must be checked again in a `while` loop after every wakeup. The readers–writers solution allowed many readers at once but exposed the risk of starvation for the writers. Finally, two mutexes taken in opposite order produced a deadlock that, without the artificial pause, appeared in only about a third of the runs, which is exactly what makes such bugs dangerous. Taking all locks in one fixed global order removes the circular wait and with it the deadlock.